

Two Books · Biology — Lecture Notes

Reading the corpus through a genome lens: letters $\approx$ bases, roots $\approx$ codons, words $\approx$ proteins. Every module carries the eight beats; beat 6 is a real Book6 number computed by the app's engine. Honest spine: the lens is a measurement frame, not a design claim — composition is governed by common letters and by length, while the genuine signal is the grammar footprint. Shuffle nulls throughout.

Module 1 — Frame — the genome lens

What it is: a deliberate analogy: read each sūra as a sequence whose 'bases' are its letters, whose 'codons' are its roots, and whose 'proteins' are its words.

Why we do it: the lens imports a ready-made toolkit (composition profiles, codon usage, adjacency bias, conditional entropy) that lets us ask precise, testable questions about the text's makeup.

How it's done: map letter $\rightarrow$ base, root $\rightarrow$ codon, then apply the same counting and shuffle-test machinery genomics uses; compare every observed quantity to a reshuffled corpus.

What we get: a vocabulary of measurable composition statistics, each with a null model — not a metaphor we admire, but numbers we can falsify.

Why it matters: without the lens we would read composition impressionistically; without the nulls we would mistake length and common-letter frequency for hidden meaning.

In the data: the whole page runs on Book6 — the same 6,236 āyahs and 1,701 roots the app analyzes — so every figure here is reproducible live.

Takeaway: the genome lens is a measurement frame, not a claim that scripture is DNA.

Bridge: the first thing any genome lens measures is base composition — which letters, how often — so we start there.

Module 2 — Base composition — the 'bases'

What it is: the frequency of each letter in a sūra, read against the corpus-wide baseline.

Why we do it: to see whether any sūra's letter mix departs from the alphabet's ordinary usage.

How it's done: count every letter, divide by the sūra's total letters, and compare each share to the corpus profile.

What we get: a per-letter percentage profile and its small deviations from baseline.

Why it matters: it sets the floor expectation: most of any composition is just the commonest letters, so 'unusual' must be measured against that, never asserted.

In the data: in al-Fatiha the single most frequent letter is l, at 19.18% of the sūra's 146 letters; deviations from baseline are small because every sūra draws on the same alphabet.

Takeaway: base composition is dominated by common letters — read deviations, not raw counts.

Bridge: once we step up from letters (bases) to roots (codons), the same 'how often' question becomes codon usage.

Module 3 — Codon usage — the Zipf curve

What it is: the distribution of how often each root (codon) is used across the corpus.

Why we do it: to characterize whether usage is even or dominated by a few heavily reused roots.

How it's done: rank roots by frequency and fit the $\log(\text{rank})$ – $\log(\text{frequency})$ line; its slope summarizes the skew.

What we get: a single slope number describing the steepness of reuse.

Why it matters: a steep Zipf slope means a small set of roots carries most of the text — a structural fact about vocabulary that any downstream count must respect.

In the data: the corpus codon-usage curve has a log-log slope of about -1.56 — the classic heavy-skew shape: a few roots are used constantly, most rarely.

Takeaway: root usage is Zipfian: a handful of codons dominate, a long tail barely appears.

Bridge: usage tells us how often each codon appears; the next question is whether codons prefer certain neighbors — di-codon bias.

Module 4 — Di-codon bias — adjacency structure

What it is: whether specific adjacent root pairs occur more or less than chance.

Why we do it: to detect the grammatical 'wiring' that makes some roots sit next to each other.

How it's done: tally observed adjacent root pairs, compare to a shuffle of the same roots, and summarize the gap with a chi-square-like statistic and a permutation p-value.

What we get: a structure p-value: how surprising the observed adjacencies are under the null.

Why it matters: significant di-codon bias is a genuine signal of grammar — fixed phrases and collocations — distinct from mere composition.

In the data: the di-codon structure test returns $p \approx 0.005$: adjacent root pairs are significantly non-random, the footprint of grammar and fixed expressions.

Takeaway: roots are not shuffled beads — adjacency carries a real, testable grammatical footprint.

Bridge: adjacency is one kind of structure; another is how much vocabulary a sūra packs per its length — sequence complexity.

Module 5 — Sequence complexity — richness vs length

What it is: lexical richness, the ratio of distinct roots to total root-tokens in a sūra.

Why we do it: to quantify vocabulary variety while staying honest about the length confound.

How it's done: divide unique roots by total root-tokens per sūra, then check richness against $\log(\text{length})$.

What we get: a richness score per sūra and the correlation that exposes the length effect.

Why it matters: longer texts inevitably repeat vocabulary, so richness falls with length; calling a long sūra 'less rich' as if it were a choice would be a confound error.

In the data: al-Fatiha uses 18 distinct roots across 23 root-tokens — richness 0.783 — and across sūras richness correlates strongly negatively with length, exactly the repeat-with-length effect.

Takeaway: richness is mostly a length artifact — always read it against sūra size.

Bridge: if length and composition shape each sūra's profile, sūras should cluster by those very traits — which the dendrogram tests.

Module 6 — Composition clustering — sūra as a vector

What it is: representing each sūra as a vector of its usage of the top roots, then grouping similar vectors.

Why we do it: to see whether sūras fall into natural composition families.

How it's done: build the top-root usage vector per sūra and run Ward hierarchical clustering into a dendrogram.

What we get: a tree whose branches group sūras with similar composition.

Why it matters: if clusters track only style and length, that warns us composition similarity is not evidence of hidden thematic coding.

In the data: computed on Book6 top-root vectors, the clusters largely track style and length — long Medinan sūras separate from short Meccan ones — not a secret code.

Takeaway: sūras cluster by style and length, the mundane drivers of composition.

Bridge: clustering looks across sūras; zooming back into the letter stream, we ask how predictable the next letter is — Markov memory.

Module 7 — Markov memory — conditional entropy

What it is: the uncertainty of the next letter given the previous one or more letters.

Why we do it: to measure how much local context constrains the text — the script's short-range memory.

How it's done: estimate H_0 (next letter alone) and H_1 (next letter given the previous), in bits, from the corpus.

What we get: a pair of entropy values whose drop quantifies how much one letter predicts the next.

Why it matters: a real drop from H_0 to H_1 is the hallmark of natural language structure, separating script from random letter draws.

In the data: conditional entropy falls from $H_0 \approx 4.086$ bits to $H_1 \approx 3.525$ bits — knowing one previous letter removes about half a bit of uncertainty, the expected language footprint.

Takeaway: each letter genuinely predicts the next — short-range memory is real and measurable.

Bridge: composition, adjacency, and memory are now all on the table; the synthesis pulls them into one honest verdict.

Module 8 — Synthesis — what the genome lens shows

What it is: the combined reading of composition, codon usage, di-codon bias, richness, clustering, and Markov memory.

Why we do it: to state plainly what the lens licenses and what it does not.

How it's done: weigh each result against its null: which effects survive shuffling, which are mere consequences of length and common-letter frequency.

What we get: one verdict: a genuine grammar footprint (di-codon bias, $H_0 \rightarrow H_1$ drop) sitting on top of composition that is otherwise driven by letter frequency and length.

Why it matters: it inoculates against over-reading — the lens reveals ordinary language structure, not a hidden biological code.

In the data: di-codon $p \approx 0.005$ and the H_0 4.086 $\rightarrow$ H_1 3.525 drop are the real, null-beating signals; Zipf slope -1.56 and the richness–length correlation are the confounds to read against.

Takeaway: the genome lens measures real grammar, but composition is governed by common letters and length — say only what the data licenses.

Bridge: next week's FDR Summary collects every Two Books test — Disjoint Letters, Signal, Biology — into one corrected dashboard so no single p-value is read in isolation.